Advanced Binary Tree Algorithms: Next Pointer Linkage, Diameter, Vertical Order Traversal, and Top View — A Formal Unified Analysis

Abstract

This paper presents a formal, unified algorithmic analysis of four advanced binary tree problems covered in a classroom session on May 18, 2026: (1) Populating next right pointers in a perfect binary tree using level-order BFS traversal with an $O(1)$-space optimization, (2) computing the diameter of a binary tree via a single-pass post-order height decomposition, (3) performing vertical order traversal using BFS with column-index tracking, and (4) extracting the top view of a binary tree using first-seen BFS column semantics. We prove correctness for all four algorithms, derive their time and space complexities, and establish a unified framework showing that BFS with lightweight per-node state is the universal primitive underlying three of the four problems, while DFS post-order serves as the optimal strategy for the diameter. Comparative complexity tables are provided for all algorithm variants, and connections to real-world applications in compiler AST analysis, database B-tree indexing, network topology visualization, and distributed system peer communication are discussed.

Index Terms

binary trees, algorithms, data structures, computational complexity, tree traversal, recursive algorithms, formal analysis

1 Introduction

Binary trees are among the most versatile and widely studied data structures in computer science. Their hierarchical structure enables efficient representation of hierarchical data, recursive decompositions, and ordered relationships. This paper presents a formal unified analysis of four advanced binary tree algorithms derived from classroom instruction on May 18, 2026:

1) Populating Next Right Pointers in a perfect binary tree (Level-order pointer linkage)
2) Diameter of a Binary Tree (Longest path via height decomposition)
3) Vertical Order Traversal (Column-wise node grouping)
4) Top View of a Binary Tree (Leftmost node per vertical level)

All four problems share a unifying theme: they require controlled traversal strategies—level-order (BFS) or recursive depth-first—that exploit the structural properties of binary trees. We provide formal problem definitions, algorithmic descriptions, correctness proofs, and complexity analyses. Comparative tables are included for each problem.

1.1 Preliminaries and Notation

We adopt the following standard definitions throughout this paper.

Definition 1 (Binary Tree). A binary tree T is a finite set of nodes where each node v has at most two children: a left child $v.\text{left}$ and a right child $v.\text{right}$, both of which may be null. The unique node with no parent is the root of T.

Definition 2 (Perfect Binary Tree). A binary tree T is perfect if every internal node has exactly two children and all leaf nodes reside at the same depth d. A perfect binary tree of depth d contains exactly $2^{d+1} - 1$ nodes.

Definition 3 (Height of a Node). The height $h(v)$ of a node v in a binary tree is defined recursively:

$$h(v) = \begin{cases} 0 & \text{if } v \text{ is a leaf (both children null)} \\ 1 + \max(h(v.\text{left}), h(v.\text{right})) & \text{otherwise} \end{cases}$$

Definition 4 (Level of a Node). The level of the root is defined as 0. For any other node v with parent $p(v)$, $\text{level}(v) = \text{level}(p(v)) + 1$.

Definition 5 (Vertical Level (Column Index)). For a node v, define its column index $\text{col}(v)$ as follows: $\text{col}(\text{root}) = 0$. For any node v, $\text{col}(v.\text{left}) = \text{col}(v) - 1$ and $\text{col}(v.\text{right}) = \text{col}(v) + 1$.

2 Populating Next Right Pointers in a Perfect Binary Tree

2.1 Problem Statement

Given a perfect binary tree T where each node has an additional pointer next (initially null), modify T in-place so that each node's next pointer points to the next node at the same level from left to right. The rightmost node at each level should have next = null.

Formally, for each node v at level ℓ, if there exists a node u immediately to the right of v at the same level, then $v.\text{next} \leftarrow u$; otherwise $v.\text{next} \leftarrow \text{null}$.

2.2 Data Structure

```
class TreeLinkNode {
   int val;
   TreeLinkNode left;
   TreeLinkNode right;
   TreeLinkNode next;  // initially null
}
```

2.3 Approach I: Level-Order Traversal with Queue

The most natural approach uses a queue-based BFS. At each level, we process all nodes in the queue and link each node's next pointer to the subsequent node in the same level.

Algorithm (BFS Next-Pointer Linkage):

```
void connect(TreeLinkNode root) {
   if (root == null) return;
   Queue<TreeLinkNode> q = new ArrayDeque<>();
   q.offer(root);

   while (!q.isEmpty()) {
      int size = q.size();
      TreeLinkNode prev = null;

      for (int i = 0; i < size; i++) {
         TreeLinkNode currNode = q.poll();

         if (prev != null) {
            prev.next = currNode;
         }

         if (currNode.left != null)  q.offer(currNode.left);
         if (currNode.right != null) q.offer(currNode.right);

         prev = currNode;
      }
      // prev.next remains null (rightmost node)
   }
}
```

Complexity: Time $O(n)$, Space $O(n)$ for the queue (at most $n/2$ nodes at the deepest level of a perfect binary tree).

2.4 Approach II: O(1) Space Using Existing Next Pointers

Since T is a perfect binary tree, once we have established next-pointer links at level ℓ, we can use those links to traverse level ℓ and establish links at level $\ell + 1$ without any auxiliary storage.

```
void connectO1(TreeLinkNode root) {
   TreeLinkNode levelStart = root;

   while (levelStart != null && levelStart.left != null) {
      TreeLinkNode curr = levelStart;

      while (curr != null) {
         // Connect left child to right child
```

```
            curr.left.next = curr.right;

            // Connect right child to next node's left child
            if (curr.next != null) {
                curr.right.next = curr.next.left;
            }

            curr = curr.next;
        }
        levelStart = levelStart.left;
    }
}
```

Complexity: Time $O(n)$, Space $O(1)$ (no auxiliary data structure needed).

2.5 Correctness Proof

Theorem 6 (Correctness of BFS Next-Pointer Assignment). The BFS-based algorithm correctly sets v.next to the immediate right neighbor of v at the same level for every node $v \in T$, and sets v.next = null for the rightmost node at each level.

Proof. By invariant: after processing the ℓ-th iteration of the outer while loop, all nodes at level ℓ have their next pointers correctly set.

Base Case ($\ell = 0$): The root is the only node at level 0. prev is initialized to null; since there is only one node, prev.next is never set, so root.next = null. ✓

Inductive Step: Assume the invariant holds for level $\ell - 1$. At the start of the ℓ-th iteration, the queue contains exactly all 2^ℓ nodes of level ℓ in left-to-right order (guaranteed by BFS). The inner loop iterates through them in order. For each consecutive pair (v_i, v_{i+1}), we set v_i.next = v_{i+1}. For the last node v_{2^ℓ}, prev is set to it but no further assignment occurs, so v_{2^ℓ}.next remains null. ✓

By induction over all $d + 1$ levels, the algorithm is correct. □

Theorem 7 (Correctness of $O(1)$-Space Algorithm). For a perfect binary tree, the $O(1)$-space algorithm correctly populates all next pointers.

Proof. The key invariants exploited are:

1) Every internal node has exactly two children (perfectness).
2) When processing a node v at level ℓ, v.next is already set correctly (established in the previous pass).

At each node v with children v.left and v.right: v.left.next = v.right (siblings). If v.next ≠ null, then v.right.next = v.next.left (cross-parent connection). These two assignments cover all intra-level connections at level $\ell + 1$. □

TABLE 1
Comparison of Next-Pointer Algorithms

Algorithm	Time	Space	Tree Type
BFS with Queue	$O(n)$	$O(n)$	Any Binary Tree
$O(1)$-Space Next-Link	$O(n)$	$O(1)$	Perfect Binary Tree Only

3 Diameter of a Binary Tree

3.1 Problem Statement

The diameter of a binary tree T is the length (in number of edges) of the longest path between any two leaf nodes. The path need not pass through the root.

Formally:

$$\text{diam}(T) = \max_{u,v \in \text{leaves}(T)} \text{dist}(u, v)$$

where $\text{dist}(u, v)$ is the number of edges on the unique path from u to v.

3.2 Key Insight: Height Decomposition

For any node v, the longest path passing through v has length:

$$\text{diam}_v = h(v.\text{left}) + h(v.\text{right}) + 2$$

where we define $h(\text{null}) = -1$ (so a single leaf contributes height 0 to its parent's sum).

The diameter of the entire tree is:

$$\text{diam}(T) = \max_{v \in T} \left(h(v.\text{left}) + h(v.\text{right}) + 2 \right)$$

3.3 Algorithm: Single-Pass Recursive Height Computation

```
int maxDiameter = 0;  // global accumulator

int maxDepth(TreeNode root) {
    if (root == null) return -1;

    int left  = maxDepth(root.left);
    int right = maxDepth(root.right);

    // Update global diameter via current node
    maxDiameter = Math.max(maxDiameter, left + right + 2);

    // Return height of current subtree
    return Math.max(left, right) + 1;
}

int diameterOfBinaryTree(TreeNode root) {
    maxDepth(root);
    return maxDiameter;
}
```

Complexity: Time $O(n)$ (each node visited once), Space $O(h)$ for the recursive call stack where h is the height of the tree.

3.4 Correctness Proof

Theorem 8 (Correctness of Height-Based Diameter). The function maxDepth correctly computes the diameter of a binary tree in $O(n)$ time.

Proof. Let $P^* = (u, \ldots, v)$ be the longest path in T with length $\mathrm{diam}(T)$. Every path in a tree passes through a unique "apex" node v^*—the deepest common ancestor of its two endpoints. At the apex:

$$|P^*| = h(v^*.\text{left}) + h(v^*.\text{right}) + 2$$

By the post-order traversal, when v^* is processed, both $h(v^*.\text{left})$ and $h(v^*.\text{right})$ have already been computed correctly (by the inductive hypothesis on subtrees). Therefore, the expression left + right + 2 at v^* equals $|P^*|$. Since we take the global maximum, maxDiameter = $\mathrm{diam}(T)$. □ □

Lemma 9 (Return Value Correctness). For any node v, maxDepth(v) returns $h(v)$, the height of v.

Proof. By structural induction. Base case: maxDepth(null) $= -1 = h(\text{null})$. Inductive step: assuming correctness for children, $\max(\text{left}, \text{right}) + 1 = \max(h(v.\text{left}), h(v.\text{right})) + 1 = h(v)$. □ □

3.5 Example

Consider the tree with root 1, left child 2, right child 3, left-left child 4, left-right child 5:

$$\text{Inorder: } 4 - 2 - 5 - 1 - 3$$

$$h(4) = 0,\ h(5) = 0,\ h(2) = 1,\ h(3) = 0$$

$$\text{diam at node 2: } 0 + 0 + 2 = 2 \text{ (path: } 4 \to 2 \to 5)$$

$$\text{diam at node 1: } 1 + 0 + 2 = 3 \text{ (path: } 4 \to 2 \to 1 \to 3)$$

$$\mathrm{diam}(T) = 3$$

TABLE 2
Diameter Algorithm Complexity Comparison

Approach	Time	Space	Notes
Naive: all pairs BFS	$O(n^2)$	$O(n)$	Compute $\mathrm{dist}(u, v)$ for all pairs
Two-pass DFS (LCA)	$O(n)$	$O(h)$	BFS from any node, then DFS
Single-pass height	$O(n)$	$O(h)$	Optimal (this paper)

4 Vertical Order Traversal

4.1 Problem Statement

Given a binary tree T, output nodes grouped by their column index $\text{col}(v)$ from the leftmost to rightmost column. Within the same column, nodes should appear in order of their level (top to bottom). Ties within the same column and level may be broken by value.

4.2 What Must Be Tracked During Traversal

The key insight is that each node must carry two pieces of state: its column index and its level (row). We maintain these as a pair $(col, level)$.

```
class Pair {
   TreeNode node;
   int col;
   int level;
}
```

4.3 Algorithm: BFS with Column Mapping

```
List<List<Integer>> verticalOrder(TreeNode root) {
   if (root == null) return new ArrayList<>();

   // Map from column -> list of node values (in level order)
   TreeMap<Integer, List<Integer>> map = new TreeMap<>();

   Queue<Pair> q = new ArrayDeque<>();
   q.offer(new Pair(root, 0, 0));

   int minLevel = Integer.MAX_VALUE;
   int maxLevel = Integer.MIN_VALUE;

   while (!q.isEmpty()) {
      Pair curr = q.poll();
      TreeNode node = curr.node;
      int col = curr.col;
      int level = curr.level;

      map.computeIfAbsent(col, k -> new ArrayList<>())
        .add(node.val);

      minLevel = Math.min(minLevel, col);
      maxLevel = Math.max(maxLevel, col);

      if (node.left != null)
         q.offer(new Pair(node.left, col - 1, level + 1));
      if (node.right != null)
         q.offer(new Pair(node.right, col + 1, level + 1));
   }

   List<List<Integer>> result = new ArrayList<>();
   for (int i = minLevel; i <= maxLevel; i++) {
      result.add(map.get(i));
   }
   return result;
}
```

Complexity: Time $O(n \log n)$ (due to TreeMap insertions), Space $O(n)$.

4.4 Correctness Proof

Theorem 10 (Correctness of Vertical Order Traversal). The BFS-based vertical order traversal correctly groups nodes by column index in left-to-right order, with nodes within each column listed in top-to-bottom (level-ascending) order.

Proof. Column correctness: We maintain the invariant that col(root) = 0, and for any node v, col(v.left) = col(v) − 1 and col(v.right) = col(v) + 1. This directly matches the definition of the column index, so the mapping is exact.

Level ordering within columns: BFS explores nodes level by level. Within the BFS queue, nodes at level ℓ are always dequeued before nodes at level $\ell + 1$. Since we append to map[col] during dequeue, nodes within the same column appear in increasing level order. ✓

Column ordering in result: Using a TreeMap (sorted by key) ensures that columns are enumerated from minimum to maximum column index. ✓ □

4.5 Column Coordinate Analysis

Lemma 11 (Column Range). For a binary tree T with n nodes, the range of column indices is $[-n + 1, n - 1]$.

Proof. In the worst case (a skewed tree), each node moves one column to the left (or right) from its parent. Starting from column 0 at the root with n nodes, the maximum deviation in either direction is $n - 1$. □

TABLE 3
Vertical Order Traversal Algorithm Comparison

Method	Time	Space	Level-Order?
DFS + sorting	$O(n \log n)$	$O(n)$	No (needs sort by level)
BFS + HashMap	$O(n \log n)$	$O(n)$	Yes (BFS guarantees level order)
BFS + TreeMap	$O(n \log n)$	$O(n)$	Yes (and sorted columns)

5 Top View of a Binary Tree

5.1 Problem Statement

The top view of a binary tree T is the set of nodes visible when the tree is viewed from directly above. Formally, for each column index c, the top view contains the node at column c with the minimum level (i.e., the first node encountered in that column during a BFS traversal).

5.2 Relationship to Vertical Order Traversal

Top View is a special case of Vertical Order Traversal: for each column, retain only the first (topmost) node. This makes the BFS approach particularly elegant, since BFS guarantees that the first time a column is visited, it is the node at the minimum level.

5.3 Algorithm

```
List<Integer> topView(TreeNode root) {
    if (root == null) return new ArrayList<>();

    // Map from column -> first seen node value (top view)
    TreeMap<Integer, Integer> map = new TreeMap<>();

    Queue<Pair> q = new ArrayDeque<>();
    q.offer(new Pair(root, 0, 0));

    while (!q.isEmpty()) {
        Pair curr = q.poll();
        int col = curr.col;

        // Only record if this column has not been seen yet
        if (!map.containsKey(col)) {
            map.put(col, curr.node.val);
        }

        if (curr.node.left != null)
            q.offer(new Pair(curr.node.left, col - 1, curr.level + 1));
        if (curr.node.right != null)
            q.offer(new Pair(curr.node.right, col + 1, curr.level + 1));
    }
```

```
    return new ArrayList<>(map.values());
}
```

Complexity: Time $O(n \log n)$, Space $O(n)$.

5.4 Correctness Proof

Theorem 12 (Correctness of Top View Algorithm). The BFS-based top view algorithm correctly outputs the first visible node from above for each column index, in left-to-right order.

Proof. Top-node correctness: BFS processes nodes in non-decreasing level order. The first time column c is encountered during BFS, the corresponding node has the minimum level among all nodes with column index c. Since we use a containsKey check and only insert on first encounter, the map correctly stores the topmost node per column.

No undercount: Every column index that exists in T must be discovered by the BFS (since BFS visits all nodes). Therefore, no column is missed.

Left-to-right ordering: The TreeMap sorts by column index, so the final list is in left-to-right order. □ □

5.5 Comparison: Top View vs. Bottom View

TABLE 4
Top View vs. Bottom View vs. Vertical Order

Problem	Per Column	BFS or DFS	Space
Top View	First (min level) node	BFS preferred	$O(n)$
Bottom View	Last (max level) node	BFS preferred	$O(n)$
Vertical Order	All nodes (level sorted)	BFS preferred	$O(n)$

6 Unified Analysis: BFS as the Universal Binary Tree Primitive

The four algorithms analyzed in this paper are unified by a single traversal primitive: Breadth-First Search (BFS) with level tracking. We formalize this connection.

Corollary 13 (BFS as Universal Primitive). The following binary tree problems all reduce to level-order BFS traversal with additional per-node state:

- Next Pointer Linkage: Each node tracks its predecessor at the same level. BFS ensures nodes in the same level are processed consecutively.
- Diameter: Although solved with DFS, the key insight—height computation via post-order traversal—is equivalent to a bottom-up BFS. The diameter at each node equals the sum of heights of its subtrees.
- Vertical Order Traversal: BFS with per-node column tracking collects all nodes by column in level order.
- Top View: BFS with first-seen-column semantics extracts the topmost node per column.

TABLE 5
Summary of All Four Algorithms

Problem	Traversal	Time	Space	Extra State
Next Pointer (BFS)	BFS	$O(n)$	$O(n)$	Level size, prev pointer
Next Pointer ($O(1)$)	Iterative	$O(n)$	$O(1)$	levelStart pointer
Diameter	DFS (post-order)	$O(n)$	$O(h)$	Global max, heights
Vertical Order	BFS	$O(n \log n)$	$O(n)$	Column index, level
Top View	BFS	$O(n \log n)$	$O(n)$	Column index, first-seen

7 Conclusion

This paper presented a formal, unified treatment of four advanced binary tree algorithms—Next Pointer Linkage, Diameter Computation, Vertical Order Traversal, and Top View—all arising from a single classroom session on binary tree traversal techniques.

The key contributions are:

- Formal correctness proofs for all four algorithms, establishing their soundness under rigorous mathematical analysis.
- A $O(1)$-space algorithm for next-pointer linkage in perfect binary trees, exploiting the structural guarantee of perfectness.

- A single-pass diameter algorithm that avoids redundant height computations by integrating the global diameter update into a post-order DFS.
- A unified BFS framework showing that vertical order traversal and top view reduce to BFS with lightweight per-node column state.
- Complexity tables enabling principled algorithm selection under different space-time constraints.

The algorithms have direct applications in compiler syntax tree analysis (diameter for expression depth), database indexing (vertical order for B-tree column scans), network topology visualization (top view for hierarchical network displays), and distributed systems (next-pointer linkage for peer-to-peer level communication protocols).

Future work includes: extending the $O(1)$-space next-pointer algorithm to general (non-perfect) binary trees using more sophisticated threading, designing cache-oblivious variants of vertical order traversal for large trees that exceed main memory, and analyzing amortized complexity of dynamic tree diameter updates under node insertion and deletion.